K-ORCH: orquestrador para instâncias multikernel distribuídas

Eduardo Kaluf[1], **Carlos Alberto Maziero**[2]

[1] Departamento de Informática – Universidade Federal do Paraná (UFPR)
Curitiba – PR – Brazil

eduardo@kaluf.com.br, maziero@inf.ufpr.br

Abstract. *The sharp increase in the number of cores per processor, combined with the growing heterogeneity of the hardware in use, ends up exposing the trade-offs and limitations of traditional monolithic operating system architectures, visible in memory hierarchies, interconnects, and I/O configurations, as well as in the management of internal data and control structures.*
One line of research proposes, as an alternative, running multiple instances of a subset of kernels, with different configurations, on the same physical processor, all of them operating much like a distributed system. This approach to the OS is now known as the multikernel, and it has been shown to be viable as a solution that goes beyond theory.
However, orchestrating distributed nodes that use the multikernel architecture remains a challenge, due to the inherent complexity of dealing with partitioned processors and to the relative novelty of the architecture. This work proposes the development of an orchestrator specifically designed for multikernel environments, making it possible to extend the architecture beyond a single local machine, with dispatching, load balancing, and monitoring of the active instances in the environment.

Resumo. *O grande aumento da quantidade de núcleos por processador, em conjunto com a crescente heterogeneidade do hardware utilizado, acaba por expor contrapartidas e limites das arquiteturas tradicionais de sistemas operacionais monolíticos, visíveis nas hierarquias de memória, interconexões e configurações de E/S, além do gerenciamento das estruturas de dados e de controle internas.*
Uma linha de pesquisa apresenta como alternativa executar múltiplas instâncias de um subconjunto de kernels, com diferentes configurações, em um mesmo processador físico, onde todos funcionam de forma similar a um sistema distribuído. Essa maneira de tratar o SO hoje é conhecida como multikernel e já se demonstrou viável como uma solução que vai além apenas da teoria.
Porém, a orquestração de nós distribuídos que utilizam a arquitetura multikernel ainda é um desafio, pela complexidade inerente de lidar com processadores particionados e pela natureza recente da arquitetura. Este trabalho propõe o desenvolvimento de um orquestrador específico para ambientes multikernel, dando a possibilidade de evoluir a arquitetura para além de uma máquina local, com despacho, balanceamento e monitoramento das instâncias ativas no ambiente.

1. Introdução

A crescente quantidade de núcleos nos multiprocessadores é algo que vem acontecendo há um longo tempo. Os sistemas operacionais, porém, não evoluíram para tirar total proveito desses multiprocessadores, que já se consolidaram comercialmente, sendo muitas vezes necessária a criação de novas políticas, como xxxx, e de *workarounds* a fim de extrair proveito dos núcleos. Tudo isso vem com um custo lógico de implementação, complexidade de sistema, aumento da superfície do kernel e *overhead* de controle lógico. Por esses motivos, a arquitetura multikernel foi criada com o objetivo de retirar essas responsabilidades de um kernel monolítico centralizado e subdividir fisicamente os núcleos.

Duas maneiras de subdividir logicamente esses núcleos são amplamente adotadas hoje em dia. Containers são unidades executáveis de código que empacotam uma aplicação e suas dependências, tirando proveito da virtualização do SO, na qual recursos como *namespaces* e *cgroups* (Linux) podem ser utilizados para subdividir memória, disco e quantidade de CPU. A outra alternativa são as máquinas virtuais, que utilizam outra forma de virtualização, na qual um *hypervisor* realiza a comunicação entre os sistemas operacionais hóspedes e o hospedeiro, dando flexibilidade às aplicações que podem ser executadas naquela máquina. Contudo, ambas as soluções têm contrapartidas: enquanto as máquinas virtuais garantem alto isolamento entre os SOs hospedados, elas possuem um *overhead* significativo na comunicação destes com o kernel que os hospeda, por conta da tradução do *hypervisor*; os containers, por sua vez, são unidades muito mais leves, mas possuem menor isolamento de segurança e podem ser *noisy neighbors*.

Nesse contexto, a arquitetura multikernel tem o objetivo de reduzir esses problemas na origem: com kernels executando em núcleos fisicamente separados, o multikernel garante ainda mais isolamento que as máquinas virtuais, sem o *overhead* de tradução do *hypervisor*. O compartilhamento do hardware é geralmente feito por um kernel pai, que utiliza dispositivos virtuais nos kernels trabalhadores, juntamente com a comunicação por um sistema de troca de mensagens e por regiões específicas de memória compartilhada.

Atualmente, as soluções de containers e máquinas virtuais estão fortemente interligadas a sistemas distribuídos. A adoção dessas tecnologias veio em conjunto com ferramentas de orquestração poderosas, como o Kubernetes, possibilitando sua ampliação para sistemas de grande porte. Do mesmo modo, a arquitetura multikernel tem grande potencial em ambientes distribuídos, como *fog computing* e nuvem; contudo, atualmente as aplicações multikernel estão restritas a uma única máquina física, por conta da falta de um orquestrador que realize a administração dessas instâncias.

Assim, seria necessária a criação de um orquestrador capaz de suprir as especificidades dessa arquitetura emergente em ambientes distribuídos, tornando-a uma solução viável para situações em que o isolamento das VMs e a leveza dos containers, juntos, sejam de extrema importância para a aplicação em execução.

Ao longo deste trabalho, serão apresentadas as justificativas para a criação de um orquestrador multikernel (Seção 2), será formulada e explorada a pergunta central da pesquisa (Seção 2.1) e serão citados os objetivos geral e específicos desse novo orquestrador (Seção 3). Em seguida, a Seção 4 mostrará a evolução da arquitetura e os trabalhos relacionados à sua evolução para ambientes distribuídos, e a Seção 10 apresentará as abordagens e o nível da pesquisa. Por último, serão especificados os resultados esperados ao

concluir a construção do orquestrador K-ORCH (Seção 11) e um planejamento de dois anos para a realização do trabalho (Seção 12).

2. Justificativa e Motivação

A necessidade de uma terceira via se torna cada vez mais necessária em meio a problemas recorrentes que podem ser observados em ambientes de computação multi-tenant como escapes de containers (CVE-2019-5736, CVE-2024-21626), estes necessitam do baixo overhead de containers, mas ficam sujeitos a problemas de segurança por conta do baixo isolamento e em arquiteturas que necessitam de maior isolamento mas que não conseguem fugir de um overhead de até xxx% (citação) das VMs Além disso, alternativas que visam unir as benefícios dessas duas soluções já existem e são buscadas cosntantemente pela indústria, como Firecracker (Citação) e Kata Containers (Citação), ferramentas que querem exatamente o isolamente de VM com densidade de containers Sendo assim, o multikernel emerge como uma solução ótima, oferecendo ambos soluções a nível de hardware sem tradeoffs entre eles.

Essa arquitetura poderia além de poder ser muito bem aproveitada em ambientes HPC e na cloud, também funcionaria bem em edge computing, onde é necessário uma velocidade maior de entrega que é prejudicada pelo overhead das máquinas virtuais. Multikernels seriam uma vantagem enorme para otimizar aplicações de larga escala em que cojnfigurações específicas do SO influenciam diretamente no desempenho da instância sendo executada, como em bancos de dados e monitores, isso permite que não seja necessário alocar uma máquina inteira especificamente para as aplicações, evitando custos e dando maior velocidade a computação existente nestes sistemas. Além disso, uma proposta recente de multikernel utilizando Linux (RFC, 2025) aproxima essa arquitetura do ambiente mainstream, dando maior visibilidade ao assunto, contudo, sem uma ferramenta operacional de orquestração isso tudo tende a ficar restrito apenas à utilização na pesquisa e teórica, com uma adoção maior, mais visibilidade e investimentos podem impulsionar a melhorias e pesquisas de grande porte no assunto.

Uma opção seria integrar alguma arquitetura multikernel diretamente com algum orquetrador sólido e difundido como kubernetes, porém, dado as especificidades como o ciclo de vida das instâncias de multikernel serem diferentes, as partições serem específicamente feitas em hardware e da alocação depender de núcleo físicos, um orquestrador que trabalhe especificamente em cima dessas características conseguirá ter maior controle e desempenho, além da flexibilidade ao projetar a arquitetura de um novo ponto.

2.1. Pergunta de Pesquisa

A pergunta central que funciona como base desse trabalho é como orquestrar instâncias multikernel em múltiplos nós distribuidos, quais são os mecanismos necessários para provisionar, alocar e monitorar essas instâncias e qual é o custo desse gerenciamento em relação a máquinas virtuais e containers orquestrados pelo estado da arte atual.

3. Objetivos

O objetivo geral desse trabalho é projetar, implementar e analisar o K-ORCH o qual deve realizar a orquestração de nós distribuídos que utilizam a arquitetura multikernel.

3.1. Objetivo especificos

- Levantar o estado da arte: identificar o estado da arte dos orquestradores atuais e as bases de seu funcionamento, relacionando-os com a arquitetura multikernel e aprofundando a conexão entre as duas áreas, de modo a determinar quais mecanismos podem ser reaproveitados e quais precisam ser concebidos especificamente para o K-ORCH.
- Definir a unidade orquestrável: estudar e aprofundar a implementação multikernel que será utilizada pelo orquestrador, formalizando o que compõe uma instância (núcleos dedicados, região de memória, imagem e configuração do kernel) e o modelo de estado desejado que o orquestrador deverá manter.
- Projetar a arquitetura: planejar o funcionamento do plano de controle e sua interação com o plano de dados, projetar o agente responsável por instanciar as unidades multikernel a partir do kernel hospedeiro de cada nó e definir a comunicação entre os nós distribuídos.
- Implementar o ciclo de vida e o posicionamento das instâncias: implementar a criação, inicialização, parada e remoção das instâncias, bem como o mecanismo de posicionamento (*placement*), responsável por escolher o nó que atenderá cada requisição com base nos núcleos livres, na memória disponível e na topologia NUMA, distribuindo a carga entre os nós.
- Desenvolver o modelo de particionamento de recursos: implementar um modelo híbrido, no qual uma região de memória é reservada na inicialização de cada nó e os núcleos são particionados dinamicamente a cada requisição, mantendo o particionamento estático, com partições predefinidas, como referência de comparação.
- Implementar o monitoramento das instâncias: coletar métricas de estado e de uso de recursos de cada instância e nó, detectar falhas, como *kernel panic* ou perda de comunicação com o agente, e acionar a recuperação, reiniciando ou reposicionando a instância afetada.
- Avaliar experimentalmente o orquestrador: medir o tempo de provisionamento, o *overhead* do agente e do plano de controle, a densidade de instâncias por nó e a interferência de desempenho entre instâncias, comparando os resultados com máquinas virtuais e microVMs orquestradas, usando containers como referência de densidade, e comparando os modelos de particionamento estático e dinâmico.

4. Trabalhos Relacionados

Os trabalhos relacionados desta proposta abrangem três principais eixos para a elaboração do orquestrador: o histórico da arquitetura multikernel, conceitos importantes em sistemas distríbuidos (observabilidade, isolamento, disponibilidade...) e orquestração em si. O objetivo é traçar uma linha clara de onde a arquitetura multikernel se encontra e como ela se relaciona com o restante das tecnologias da atualidade, identificando a lacuna principal que o orquestrador K-ORCH irá solucionar.

5. Arquiteturas com múltiplos kernels

A ideia de estruturar o SO como um conjunto de kernels independentes surgiu há algumas décadas. O *Hive* [Chapin et al. 1995] organiza o SO como um sistema distribuído interno de kernels, que são denominados células, onde cada uma delas é responsável por um conjunto fixo de nós e opera como um SO independente, enquanto cooperam para oferecer

às aplicações uma imagem única e transparente de sistema. A comunicação entre células ocorre por meio de *remote procedure calls* (RPC) e por leitura direta da memória de outras células.

Em vez de reescrever o SO por completo, *Disco* [Bugnion et al. 1997] ataca o problema inserindo um pequeno monitor de máquinas virtuais que executa várias cópias de um SO na mesma máquina, comunicando-se por protocolos distribuídos comuns como NFS e TCP/IP. A abordagem permite executar sistemas operacionais especializados lado a lado com sistemas comerciais.

O *Cellular Disco* [Govil et al. 2000] estende a proposta dividindo o próprio monitor em partes menores, a fim de conter falhas de hardware, transformando o multiprocessador no que seria um cluster virtual.

A formulação moderna do conceito é apresentada por [Baumann et al. 2009], que propõem tratar a máquina como uma rede de núcleos independentes. Os autores foram responsáveis por desenvolver o sistema *Barrelfish*, onde é executado um kernel por núcleo e três princípios básicos são seguidos: comunicação entre núcleos sempre é explícita, sua estrutura é neutra em relação ao hardware e o estado é replicado em vez de ser compartilhado, sendo mantido consistente por processos monitores. Em um trabalho anterior do mesmo grupo, [Schüpbach et al. 2008] argumentam que a diversidade de hardware, nas formas de não uniformidade de memória, heterogeneidade de núcleos e diversidade entre sistemas, será um desafio tão grande quanto a escalabilidade no futuro.

Mais recentemente, a proposta de suporte a múltiplas instâncias do Linux em uma mesma máquina [Wang 2025b] utiliza a ideia de carregar as instâncias via *kexec*, recebendo núcleos e memória que podem ser realocados em tempo de execução por meio de uma *device tree* e do mecanismo de *hotplug* [Wang 2025a]. A ferramenta kerf [Multikernel Technologies 2025] ficou responsável por gerenciar a criação, configuração, monitoramento e a destruição dessas instâncias.

6. Balanceamento de Carga

Decidir onde executar cada carga e como redistribuí-la conforme a demanda é um problema estudado há muito tempo em sistemas distribuídos. [Casavant and Kuhl 1988] propõem uma taxonomia hierárquica de escalonamento em sistemas distribuídos que distingue abordagens estáticas de dinâmicas, Além disso, as abordagens dinâmicas são classificadas quanto à distribuição da responsabilidade pelas decisões e à cooperação entre os nós.

[Shivaratri et al. 1992] decompõem os algoritmos de distribuição de carga em quatro principais políticas: de transferência (decidir quando mover uma tarefa), de seleção (qual tarefa mover), de localização (para onde mover) e de informação (quando coletar o estado dos nós). Os algoritmos ainda são classificados naqueles que são iniciados pelo emissor, pelo receptor ou simétricos.

Dois resultados fundamentam políticas simples, mas eficientes e escaláveis de balanceamento. Primeiro, [Mitzenmacher 2001] mostra que escolhendo a opção menos carregada entre apenas dois destinos aleatórios reduz exponencialmente a carga máxima esperada em relação a simplesmente escolher aleatoriamente. Em outro estudo, [Blumofe and Leiserson 1999] provam que o roubo de tarefas de maneira aleatorizada de

um processador ativo para outro ocioso, executa computações com tempo próximo do ótimo.

No contexto de sistemas com múltiplos kernels, desde o início, era necessário saber como distribuir os recursos da máquina entre as instâncias. No sistema Hive, os mecanismos de compartilhamento de recursos são implementados pela cooperação entre as células, mas a política é atribuída a um processo de usuário, o único com visão global do sistema, responsável por decidir de quais células alocar memória e como escalonar processos [Chapin et al. 1995].

No sistema Disco, o monitor aplica políticas globais sobre as máquinas virtuais, movendo memória entre elas para evitar paginação desnecessária e escalonando dinamicamente as vCPUs para equilibrar a carga [Bugnion et al. 1997]. O sistema Cellular Disco [Govil et al. 2000] apresenta uma solução mais elaborada para sistemas multiprocessadores. Nele, cada processador mantém sua própria fila de processadores virtuais, combinando um balanceador ocioso, que usa apenas informação local, com um balanceador periódico, que consulta uma árvore de carga global.

No sistema Barrelfish, a base de conhecimento do sistema reúne informação pelo hardware e medições em tempo de execução, respondendo a consultas de otimização, como a alocação de drivers a núcleos e a construção de árvores de mensagens adaptadas à máquina [Baumann et al. 2009, Schüpbach et al. 2008].

Nos multikernels com foco em HPC, a gerência das instâncias tende a ser exposta como uma interface. A IHK oferece operações para reservar núcleos e memória, inicializar, consultar, desligar e destruir instâncias de kernels leves, porém, não define políticas sobre quando as utilizar [Shimosawa et al. 2014].

Em um ambiente multikernel, a IHK concentra a gerência no kernel *master* [Shimosawa et al. 2014], responsável por tomar o controle do mapeamento da memória física e orquestrar a alocação e reserva de recursos para os kernels *slave* do ambiente.

7. Observabilidade

Esta seção apresenta fundamentos de como observar, coletar dados e analisá-los em sistemas distribuídos, em seguida como trabalhos aplicam técnicas parecidas em ambientes com múltiplos kernels.

[Mansouri-Samani and Sloman 1993] definem um modelo funcional de monitoramento de sistemas distribuídos dividido em quatro atividades (geração, processamento, disseminação e apresentação de informação), tratadas como blocos que se combinam em diferentes proporções e momentos da análise, sendo uma base essencial para os trabalhos que seguem. Além disso, apontam a desatualização da informação como o principal problema existente dentro do monitoramento.

Em uma revisão recente voltada a arquitetura de microsserviços, [Faseeha et al. 2025] analisam ferramentas de observabilidade em diferentes níveis, como de sistema, de serviço e de rede, organizadas em torno de três fontes que hoje já são comumente disseminadas: *metrics*, *logs* e *traces*.

Para métricas em clusters, o sistema Ganglia [Massie et al. 2004] adota uma organização hierárquica: dentro de cada cluster, os nós comunicam e trocam seus estados

via *multicast*. Clusters distintos são federados por uma árvore de conexões *peer-to-peer*, com armazenamento em bases de séries temporais. O projeto privilegia baixa sobrecarga por nó e escalabilidade, ao custo de métricas agregadas e de uma menor granularidade temporal.

O serviço *Lightweight Distributed Metric Service* (LDMS) [Agelastos et al. 2014] mostra que é possível monitorar continuamente um sistema inteiro, com a frequência adequada, mesmo em ambientes HPC. LDMS utiliza processos coletores nos nós e processos agregadores, que reúnem os dados. Os autores quantificam a sobrecarga do monitoramento sobre as aplicações como baixa.

O sistema Omni-Kernel [Kvalnes et al. 2015] mede o consumo de recursos por completo e o atribui a um processo, uma máquina virtual ou uma transação. Nele, o kernel é fatorado em componentes que se comunicam por mensagens, e escalonadores existem no meio do caminho, responsáveis por controlar a ordem de processamento e contabilizar o consumo.

Em uma proposta de hardware, [West et al. 2009] dedicam um núcleo específico ao monitoramento, alimentado por detectores de eventos distribuídos pelo processador, mostrando como uma instância de kernel poderia contabilizar e exportar seu próprio consumo.

Os próprios sistemas com múltiplos kernels também possuem sistemas de monitoramento e balanceamento próprios. A base de conhecimento do sistema Barrelfish incorpora medições em tempo de execução, como a latência de comunicação entre cada par de núcleos [Baumann et al. 2009], o balanceador periódico do sistema Cellular Disco depende de uma árvore de carga atualizada constantemente [Govil et al. 2000] e o serviço PIKA mede o tempo de cada aplicação para orientar as escolhas realizadas [Beckmann et al. 2014]. Porém, a informação é coletada e consumida numa única máquina, não sendo explorada a ideia do compartilhamento entre nós em um cluster.

8. Disponibilidade e Isolamento

Uma grande motivação para dividir o sistema em múltiplos kernels é a disponibilidade das aplicações. No sistema Hive [Chapin et al. 1995], uma falha de hardware ou de software afeta apenas os processos que usam os recursos da célula prejudicada. O sistema emprega uma proteção em cada página contra escritas de outras células, descarta de maneira preventiva as páginas que uma célula que falhou poderia ter corrompido e possui detecção de falhas por heurísticas, cuja confirmação exige consenso das células sobreviventes.

O sistema Disco trata cada máquina virtual como unidade de contenção de falhas de software [Bugnion et al. 1997], e o sistema Cellular Disco estende essa contenção a falhas de hardware. Em centenas de experimentos com injeção de falhas, a recuperação foi bem-sucedida em todos os casos, levando menos de meio segundo [Govil et al. 2000].

Além de conter falhas, a divisão em múltiplos kernels também é usada para isolar o desempenho das aplicações. O sistema Pisces [Ouyang et al. 2015] permite a coexistência, em um mesmo nó, de múltiplos ambientes de SO independentes e isolados, denominados enclaves, cada um executando um co-kernel leve, capaz de gerenciar uma partição de recursos de hardware, tratando o isolamento de desempenho como requisito principal e tendo seu foco em HPC

Com ênfase na parte de segurança, pesquisas como BlackBox [Van't Hof and Nieh 2022] buscam proteger a confidencialidade e a integridade de contêineres sem confiar no SO, por meio de um monitor que cria espaços de endereçamento físico protegidos para cada contêiner. Por outro lado, o *framework* MultiK [Kuo et al. 2019] garante maior segurança reduzindo a superfície de ataque ao remover mais de 90% do código do kernel visível a cada aplicação que está sendo executada. Esses trabalhos, por mais que não executados sob a arquitetura multikernel, ressoam fortemente com sua filosofia.

9. Orquestração

Ao revisar orquestradores como Kubernetes, Docker Swarm e Apache Mesos, [Swetha et al. 2025] identificam a escassez de propostas de orquestração orientada pela utilização de recursos e comparam a adequação de cada ferramenta. Na mesma linha, [Lu et al. 2025] propõem uma implantação de contêineres em dois estágios: um algoritmo guloso distribui contêineres entre máquinas virtuais com granularidade grossa enquanto outro algoritmo genético reparte os recursos dentro de cada uma das máquinas virtuais.

10. Metodologia

A metodologia utilizada por este trabalho terá um caráter teórico e prático, com a análise de possíveis algoritmos a fim de realizar o controle das instâncias de multikernel e então a sua prórpia implementação. Além disso, uma abordagem quantitativa será utilizada a fim de identificar os benefícios e comparar o desempenho entre aplicações utilizando o orquestrador e outras com a utilização de containers por exemplo. Por último, a ideia é realizar um trabalho de nível exploratório, testando e verificando diferentes possibilidades que possam ser utilizadas para gerenciar esse novo tipo de tecnologia.

10.1. Caracterizar e Estudar modelo selecionado

Na primeira etapa será estudado o modelo teórico geral de multikernels e então a instânciação da multikernel technologies, dessa forma, as considerações de como manusear a orquestração poderá ser feita com base na arquitetura específica que está sendo trabalhada.

10.2. Projeto e implementação do protótipo

Na segunda etapa será realizado o projeto do orquestrador e de seus agentes, tendo como objetivo ter uma prova de conceito para o orquestrador final que será implementado. Será necessário fazer o mapeamento do protótipo, casos de uso e design da aplicação como um todo.

10.3. Integração de funcionalidades avançadas

Na terceira etapa, serão melhoradas e aprofundadas pequenas funcionalidades da prova de conceito e adicionado novas como balanceamento de carga e replicação, por exemplo.

10.4. Análise quantitativa e refinamento da ferramenta

A quarta etapa consiste em relizar testes e coletar dados sobre o comportamento do orquestrador e das instâncias dos nós multikernel, permitindo uma análise mais profunda

sobre pontos fracos e fortes do sistema como um todo. A validação irá comparar o desempenho do próprio orquestrador com outros orquestradores de containers e VMs além do próprio desempenho das aplicações, com kernels configurados especificamente para cada situação, como por exemplo comparar um sistema distribuído que realiza um processamento de dados pesados e armazena resultados em um banco relacional, desta forma será possível ver a diferença de ter kernels especializados para as operações e o impacto real do overhead das máquinas virtuais.

Com esses dados coletados, uma análise será feita sob eles e modificações mais específicas no orquestrador poderão ser realizadas, a fim de melhorar seu desempenho. Ademais, uma análise do desempenho da própria arquitetura serpa realizada, que poderá ser utilizada para justifica-la como uma opção viável em diferentes casos de uso.

11. Resultados Esperados

A proposta busca entender e explorar os melhores caminhos para a especificidade da arquitetura multikernel, onde ao final do trabalho esteja disponível um orquestrador robusto para sistema distribuidos que com nós utilizando a arquitetura multikernel.

Almeja-se definir bons parametros do que se pode melhorar e onde são as maiores dificuldades para a realização da orquestração, mapear e documentar os algoritmos e ideias utilizadas a fim de se ter uma compreensão mais profunda da junção dessas duas áreas para implementações e futuros avanços da própria ferramenta, ou de ferramentas parecidas.

De modo geral, este trabalho poderá impulsionar a arquitetura multikernel como uma solução viável em sistemas distribuidos e escalaveis, dando maior visibilidade e consequentemente atrainda mais atenção para pesquisa e investimentos nesta área.

12. Cronograma

A tabela 1 apresenta o cronograma organizado em 8 trimestres. O dois primeiros tem como foco uma análise mais profunda da instância de multikernel que será utilizada, com experimentos práticos e trabalhos a fim de correlacionar seu modo de funcionamento com a futura ferramenta de orquestração. Durante o terceiro e quarto semestre a idea é projetar o funcionamento geral do orquetrador, implementando então, os agentes de orquestração e a aplicação pai que receberá as requisições em si, tendo ao final do primeiro ano uma unidade já funcional de orquestração. Nos semestres cinco e seis, será realizada a implementação de algoritmos e ferramentas que tornarão o orquestrador mais robusto, dando mais funcionalidades e possibilidades para o usuário final. Por último, nos trimestres sete e oito, será realizado testes e analises quanto ao desempenho, comparando tanto a orquestração como as próprias aplicações de multikernel em si, visando ter uma melhor noção de como a arquitetura se compartaria em escalas maiores e competitivas, provocando também mudanças mais finas de operação do orquestrador.

13. Considerações Finais

Este trabalho propõe uma ferramenta de orquestração para arquiteturas multikernel. Ao analisar diversos modos de orquestração K-ORCH pretende os adaptar com o objetivo de fornecer uma solução robusta para ambientes distríbuidos utilizando multikernel, permitindo unir desempenho e segurança.

Tabela 1. Cronograma das tarefas organizado por trimestre.

Atividade	T1	T2	T3	T4	T5	T6	T7	T8
Levantamento do estado da arte	X	X						
Caracterização experimental do modelo multikernel selecionado	X	X						
Definição da unidade orquestrável e do modelo de estado		X						
Projeto da arquitetura (plano de controle, agentes, comunicação)			X					
Implementação do ciclo de vida e do posicionamento das instâncias			X	X				
Implementação do particionamento híbrido de recursos					X	X		
Implementação do monitoramento e da recuperação de instâncias					X	X		
Avaliação experimental e comparativa (VMs e microVMs)							X	X
Refinamento do orquestrador							X	X
Redação de artigos e do texto final				X		X	X	X

A contribuição esperada é proporcionar uma terceira via para ambientes que necessitam do melhor dos containers e das máquinas virtuais, proporcionando abstração suficiente a fim de escalar a arquitetura para grandes sistemas de produção com facilidade e confiabilidade.

Referências

Agelastos, A., Allan, B., Brandt, J., Cassella, P., Enos, J., Fullop, J., Gentile, A., Monk, S., Naksinehaboon, N., Ogden, J., Rajan, M., Showerman, M., Stevenson, J., Taerat, N., and Tucker, T. (2014). The lightweight distributed metric service: A scalable infrastructure for continuous monitoring of large scale computing systems and applications. In *International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, pages 154–165, New Orleans - LA, USA.

Baumann, A., Barham, P., Dagand, P.-E., Harris, T., Isaacs, R., Peter, S., Roscoe, T., Schüpbach, A., and Singhania, A. (2009). The multikernel: A new OS architecture for scalable multicore systems. In *22nd ACM Symposium on Operating Systems Principles (SOSP)*, pages 29–44, Big Sky - MT, USA.

Beckmann, N. Z., Gruenwald III, C., Johnson, C. R., Kasture, H., Sironi, F., Agarwal, A., Kaashoek, M. F., and Zeldovich, N. (2014). PIKA: A network service for multikernel operating systems. Technical Report MIT-CSAIL-TR-2014-002, MIT Computer Science and Artificial Intelligence Laboratory, Cambridge - MA, USA.

Blumofe, R. D. and Leiserson, C. E. (1999). Scheduling multithreaded computations by work stealing. *Journal of the ACM*, 46(5):720–748.

Bugnion, E., Devine, S., Govil, K., and Rosenblum, M. (1997). Disco: Running commodity operating systems on scalable multiprocessors. *ACM Transactions on Computer Systems*, 15(4):412–447.

Casavant, T. L. and Kuhl, J. G. (1988). A taxonomy of scheduling in general-purpose distributed computing systems. *IEEE Transactions on Software Engineering*, 14(2):141–154.

Chapin, J., Rosenblum, M., Devine, S., Lahiri, T., Teodosiu, D., and Gupta, A. (1995). Hive: Fault containment for shared-memory multiprocessors. In *15th ACM Symposium on Operating Systems Principles (SOSP)*, pages 12–25, Copper Mountain - CO, USA.

Faseeha, U., Syed, H. J., Samad, F., Zehra, S., and Ahmed, H. (2025). Observability in microservices: An in-depth exploration of frameworks, challenges, and deployment paradigms. *IEEE Access*, 13:72011–72039.

Govil, K., Teodosiu, D., Huang, Y., and Rosenblum, M. (2000). Cellular Disco: Resource management using virtual clusters on shared-memory multiprocessors. *ACM Transactions on Computer Systems*, 18(3):229–262.

Kuo, H.-C., Gunasekaran, A., Jang, Y., Mohan, S., Bobba, R. B., Lie, D., and Walker, J. (2019). MultiK: A framework for orchestrating multiple specialized kernels. `https://arxiv.org/abs/1903.06889`. arXiv:1903.06889. Acessado em 10/10/2026.

Kvalnes, Å., Johansen, D., van Renesse, R., Schneider, F. B., and Valvåg, S. V. (2015). Omni-Kernel: An operating system architecture for pervasive monitoring and scheduling. *IEEE Transactions on Parallel and Distributed Systems*, 26(10):2849–2862.

Lu, C., Zhou, J., and Zou, Q. (2025). An optimized approach for container deployment driven by a two-stage load balancing mechanism. *PLOS ONE*, 20(1). Artigo e0317039.

Mansouri-Samani, M. and Sloman, M. (1993). Monitoring distributed systems. *IEEE Network*, 7(6):20–30.

Massie, M. L., Chun, B. N., and Culler, D. E. (2004). The Ganglia distributed monitoring system: Design, implementation, and experience. *Parallel Computing*, 30(7):817–840.

Mitzenmacher, M. (2001). The power of two choices in randomized load balancing. *IEEE Transactions on Parallel and Distributed Systems*, 12(10):1094–1104.

Multikernel Technologies (2025). kerf: Multikernel management system. `https://github.com/multikernel/kerf`. Acessado em 10/10/2026.

Ouyang, J., Kocoloski, B., Lange, J. R., and Pedretti, K. (2015). Achieving performance isolation with lightweight co-kernels. In *24th International Symposium on High-Performance Parallel and Distributed Computing (HPDC)*, pages 149–160, Portland - OR, USA.

Schüpbach, A., Peter, S., Baumann, A., Roscoe, T., Barham, P., Harris, T., and Isaacs, R. (2008). Embracing diversity in the Barrelfish manycore operating system. In *Workshop on Managed Many-Core Systems (MMCS)*, Boston - MA, USA.

Shimosawa, T., Gerofi, B., Takagi, M., Nakamura, G., Shirasawa, T., Saeki, Y., Shimizu, M., Hori, A., and Ishikawa, Y. (2014). Interface for heterogeneous kernels: A framework to enable hybrid OS designs targeting high performance computing on many-

core architectures. In *21st International Conference on High Performance Computing (HiPC)*, pages 1–10, Goa - India.

Shivaratri, N. G., Krueger, P., and Singhal, M. (1992). Load distributing for locally distributed systems. *Computer*, 25(12):33–44.

Swetha, R., Thriveni, J., and Venugopal, K. R. (2025). Resource utilization-based container orchestration: Closing the gap for enhanced cloud application performance. *SN Computer Science*, 6. Artigo 191.

Van't Hof, A. and Nieh, J. (2022). BlackBox: A container security monitor for protecting containers on untrusted operating systems. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 683–700, Carlsbad - CA, USA.

Wang, C. (2025a). Multikernel: Kernel-to-kernel isolation with elastic resource management. `https://lpc.events/event/19/contributions/2074/`. Linux Plumbers Conference. Acessado em 10/10/2026.

Wang, C. (2025b). [RFC patch 0/7] kernel: Introduce multikernel architecture support. `https://lore.kernel.org/lkml/20250918222607.186488-1-xiyou.wangcong@gmail.com/`. Linux Kernel Mailing List. Acessado em 10/10/2026.

West, P. E., Peress, Y., Tyson, G. S., and McKee, S. A. (2009). Core monitors: Monitoring performance in multicore processors. In *6th ACM Conference on Computing Frontiers (CF)*, Ischia - Italy.